

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 5**



SEARCHING

Oleh:

Muhammad Rifqi Rahman

NIM. 2510817310003

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
TAHUN 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 5

Laporan Praktikum Algoritma & Struktur Data Modul 5: Searching ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Rifqi Rahman
NIM : 2510817310003

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	ii
DAFTAR ISI	iii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL	v
KOMPLEKSITAS WAKTU.....	1
A. Source Code.....	1
B. Output	3
C. Linear Search.....	3
D. Binary Search	4
KARAKTERISTIK ALGORITMA	5
A. Linear Search.....	5
B. Binary Search	5
ANALISIS OPERASI	6
A. Linear Search.....	6
B. Binary Search	6
ANALISIS PERFORMA	8
A. Perbandingan waktu pada kondisi yang sama	8
B. Efisiensi untuk dataset kecil dan besar	8
C. Pengaruh kondisi data.....	8
D. Teori $O(n)$ vs $O(\log n)$ dan hasil eksperimen	9
TAUTAN GIT	10

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Linear Search	3
Gambar 2. Screenshot Hasil Binary Search.....	3

DAFTAR TABEL

Tabel 1 Source Code sorting_algorithms.cpp.....	1
---	---

KOMPLEKSITAS WAKTU

A. Source Code

Tabel 1 Source Code sorting_algorithms.cpp

1	#include "searching_algorithms.h"
2	#include <chrono>
3	
4	using Clock =
	std::chrono::high_resolution_clock;
5	
6	void linear_search(const std::vector<int>&
	data, int target, Metrics& m) {
7	auto start = Clock::now();
8	
9	m.comparisons = 0;
10	m.found_index = -1;
11	
12	for (size_t i = 0; i < data.size(); ++i) {
13	m.comparisons++;
14	if (data[i] == target) {
15	m.found_index = i;
16	break;
17	}
18	}
19	
20	m.time_us = std::chrono::duration<double,
	std::micro>(Clock::now() - start).count();
21	}
22	
23	void binary_search(const std::vector<int>&
	data, int target, Metrics& m) {

```

24     auto start = Clock::now();
25
26     m.comparisons = 0;
27     m.found_index = -1;
28
29     int left = 0;
30     int right = data.size() - 1;
31
32     while (left <= right) {
33         m.comparisons++;
34         int mid = left + (right - left) / 2;
35
36         if (data[mid] == target) {
37             m.found_index = mid;
38             break;
39         }
40
41         if (data[mid] < target) {
42             left = mid + 1;
43         } else {
44             right = mid - 1;
45         }
46     }
47
48     m.time_us = std::chrono::duration<double,
std::micro>(Clock::now() - start).count();
49 }

```

B. Output

```
>> Linear Search
```

Algorithm	Case	N	Comparisons	Found Index	Time (us)
Linear Search	Best	1000	1	0	0.166
Linear Search	Average	1000	375	374	2.663
Linear Search	Worst	1000	1000	Not Found	5.955
Linear Search	Best	10000	1	0	0.043
Linear Search	Average	10000	3746	3745	15.087
Linear Search	Worst	10000	10000	Not Found	39.798
Linear Search	Best	100000	1	0	0.045
Linear Search	Average	100000	37455	37454	171.783
Linear Search	Worst	100000	100000	Not Found	506.037
Linear Search	Best	1000000	1	0	0.068
Linear Search	Average	1000000	374541	374540	1563.503
Linear Search	Worst	1000000	1000000	Not Found	4873.866

Gambar 1. Screenshot Hasil Linear Search

```
>> Binary Search
```

Algorithm	Case	N	Comparisons	Found Index	Time (us)
Binary Search	Best	1000	9	500	0.153
Binary Search	Average	1000	3	374	0.093
Binary Search	Worst	1000	10	Not Found	0.153
Binary Search	Best	10000	13	5000	0.184
Binary Search	Average	10000	13	3745	0.197
Binary Search	Worst	10000	14	Not Found	0.159
Binary Search	Best	100000	16	50000	0.263
Binary Search	Average	100000	16	37454	0.205
Binary Search	Worst	100000	17	Not Found	0.144
Binary Search	Best	1000000	19	500000	0.371
Binary Search	Average	1000000	15	374540	0.528
Binary Search	Worst	1000000	20	Not Found	0.216

Gambar 2. Screenshot Hasil Binary Search

C. Linear Search

Linear Search memeriksa elemen satu per satu dari indeks 0 ke $n-1$, tanpa asumsi tentang urutan data. Karena tidak ada cara memotong proses kecuali saat target ditemukan, batas bawah (best case) adalah 1 comparison dan batas atas (worst case) adalah n comparison.

Best case $O(1)$ terjadi ketika target ada di indeks 0, comparisons = 1, tidak bergantung pada N . Terkonfirmasi di semua ukuran data pada eksperimen.

Average case $O(n)$ terjadi ketika target di posisi tengah. Comparisons $\approx n/2$. Data eksperimen menunjukkan comparisons $\approx 0,375n$ ($N=1.000 \rightarrow 375$, $N=10.000 \rightarrow 3.746$, $N=1.000.000 \rightarrow 374.541$), konsisten sebagai proporsi tetap dari N .

Worst case $O(n)$ terjadi ketika target tidak ada. Seluruh array harus diperiksa, comparisons = n . Waktu tumbuh $\sim 10\times$ setiap $10\times$ kenaikan N ($5,955 \rightarrow 39,798 \rightarrow 506,037 \rightarrow 4.873,866 \mu s$), mengkonfirmasi $O(n)$ secara langsung.

D. Binary Search

Binary Search memotong ruang pencarian menjadi setengah di setiap langkah. Dari N elemen, setelah k langkah sisa ruang pencarian adalah $N/2^k$. Pencarian berhenti ketika $N/2^k < 1$, yaitu $k > \log_2 N$, sehingga batas atas comparisons adalah $\lceil \log_2(n+1) \rceil$.

Best case $O(\log n)$, bukan $O(1)$. Target di posisi tengah array ditemukan lebih cepat, namun implementasi tetap membutuhkan beberapa langkah biseksi. Comparisons = 9, 13, 16, 19 untuk $N = 1.000, 10.000, 100.000, 1.000.000$, sesuai $\lceil \log_2 n \rceil$.

Average case $O(\log n)$, target di posisi sembarang. Comparisons: 3, 13, 16, 15. Angka 3 untuk $N=1.000$ terjadi karena target kebetulan ditemukan sangat awal dalam proses biseksi, bukan berarti lebih baik dari $\log n$, tetap terbatas dalam $O(\log n)$.

Worst case $O(\log n)$, target tidak ada. Comparisons: 10, 14, 17, 20 untuk $N = 1.000, 10.000, 100.000, 1.000.000$. Kenaikan hanya +3–4 untuk setiap $10\times$ kenaikan N , sesuai $\log_2(10) \approx 3,32$. Ini adalah bukti paling bersih dari pertumbuhan logaritmik.

KARAKTERISTIK ALGORITMA

A. Linear Search

Linear Search tidak memerlukan data terurut. Tidak ada asumsi struktural yang dibuat sebelum pencarian, sehingga bekerja pada data acak, terurut, maupun terbalik tanpa perbedaan logika.

Algoritma ini bersifat iteratif: satu loop dari indeks 0 ke $n-1$. Bisa ditulis rekursif, tapi tidak ada manfaatnya karena tidak ada pembagian masalah yang terjadi, hanya menambah overhead call stack tanpa mempercepat pencarian.

Mekanisme inti: bandingkan $arr[i]$ dengan target. Jika cocok, kembalikan i . Jika i sudah di luar batas, kembalikan -1 . Satu-satunya optimasi yang valid adalah early termination saat ditemukan, yang sudah built-in dalam definisi algoritmanya.

B. Binary Search

Binary Search mutlak memerlukan data terurut. Ini bukan sekadar syarat teknis, melainkan syarat korektif: keputusan membuang setengah ruang pencarian ("target pasti di kiri mid") hanya valid jika data terurut. Pada data tidak terurut, Binary Search bisa memberikan false negative, target ada tapi tidak ditemukan.

Bisa diimplementasikan iteratif maupun rekursif. Keduanya $O(\log n)$, tapi versi iteratif lebih efisien secara memori karena tidak menambah call stack sebesar $O(\log n)$.

Mekanisme inti: hitung $mid = (left + right) / 2$, bandingkan $arr[mid]$ dengan target. Jika sama, return mid . Jika $target < arr[mid]$, geser $right = mid - 1$. Jika $target > arr[mid]$, geser $left = mid + 1$. Ulangi hingga $left > right$. Setiap iterasi membuang separuh ruang pencarian yang tersisa.

ANALISIS OPERASI

analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Linear Search

Comparisons pada Linear Search adalah fungsi langsung dari posisi target: jika target di indeks k , maka comparisons = $k+1$. Ini terlihat dari data eksperimen: $N=1.000$ average $\rightarrow$ comparisons=375, index_found=374; $N=10.000$ average $\rightarrow$ comparisons=3.746, index_found=3.745. Hubungan index_found = comparisons - 1 berlaku konsisten di semua kasus.

Pengaruh ukuran N : best case tidak terpengaruh N sama sekali (selalu 1 comparison). Average dan worst case tumbuh linear, worst case comparisons = N secara eksak, average $\approx 0,375N$ dari data eksperimen. Artinya Linear Search tidak bisa memanfaatkan pertambahan N untuk efisiensi, setiap pertambahan N langsung menambah beban pencarian secara proporsional.

B. Binary Search

Comparisons pada Binary Search tidak mencerminkan posisi langsung target. Menariknya, best case di data eksperimen (target di tengah) justru menghasilkan comparisons lebih tinggi dari average case pada $N=1.000$ (Best=9, Average=3). Ini terjadi karena "best case" didefinisikan sebagai target di indeks $N/2$ (tengah array), yang baru dicapai setelah beberapa langkah biseksi dari mid awal, sementara average case kebetulan menemukan target lebih awal.

Pengaruh ukuran N sangat kecil: kenaikan N sebesar $1.000\times$ (dari 1.000 ke 1.000.000) hanya menambah ~ 10 comparisons (dari 10 ke 20 untuk worst case). Ini adalah konsekuensi langsung $O(\log n)$: $\log_2(1.000.000) - \log_2(1.000) = 20 - 10 = 10$. Bandingkan dengan Linear Search yang comparisons-nya ikut naik $1.000\times$ untuk kenaikan N yang sama.

ANALISIS PERFORMA

A. Perbandingan waktu pada kondisi yang sama

Pada best case, Linear Search lebih cepat dari Binary Search di semua ukuran N. Linear Search hanya butuh 1 comparison (langsung cek indeks 0), sedangkan Binary Search tetap melakukan $\sim \log n$ langkah. Untuk $N=10.000$ best case: Linear= $0,043 \mu s$ vs Binary= $0,184 \mu s$. Ini bukan kelemahan Binary Search, best case keduanya memang berbeda definisinya. Best case Linear Search adalah skenario yang paling menguntungkan secara struktural (target di posisi pertama), sedangkan best case Binary Search masih dibatasi oleh struktur biseksinya.

Pada average dan worst case, Binary Search unggul makin besar seiring N. Worst case $N=1.000$: selisih $\sim 39\times$. Worst case $N=10.000$: selisih $\sim 250\times$. Worst case $N=1.000.000$: selisih $\sim 22.564\times$. Rasio ini terus membesar karena Linear Search tumbuh $O(n)$ sementara Binary Search $O(\log n)$, selisihnya adalah $O(n/\log n)$ yang terus membesar.

B. Efisiensi untuk dataset kecil dan besar

Dataset kecil ($N=1.000$): perbedaan performa keduanya masih dalam skala μs yang tidak kritis. Pilihan lebih ditentukan oleh kondisi data, jika data tidak terurut dan tidak perlu sorting, Linear Search lebih praktis karena tidak membutuhkan preprocessing.

Dataset besar ($N=1.000.000$): Binary Search worst case = $0,216 \mu s$, Linear Search worst case = $4.873,866 \mu s$. Jika pencarian dilakukan berkali-kali, overhead ini menjadi bottleneck nyata. Pada skala ini, biaya sorting untuk memungkinkan Binary Search hampir selalu sepadan, terutama jika data relatif statis.

C. Pengaruh kondisi data

Linear Search tidak terpengaruh oleh urutan data. Yang menentukan performa hanya posisi target relatif terhadap indeks 0. Data acak dan data terurut dengan target di posisi yang sama menghasilkan comparisons identik.

Binary Search sepenuhnya bergantung pada data terurut. Jika data sering berubah (insert/delete), ada overhead tambahan untuk menjaga keurutan. Ini adalah trade-off yang perlu dipertimbangkan: Binary Search efisien untuk pencarian berulang pada data statis, tapi

kurang ideal untuk data yang sangat dinamis tanpa struktur data yang tepat (misalnya balanced BST).

D. Teori $O(n)$ vs $O(\log n)$ dan hasil eksperimen

Konfirmasi $O(n)$ Linear Search: waktu worst case untuk $N = 1.000, 10.000, 100.000, 1.000.000$ adalah $5,955 \rightarrow 39,798 \rightarrow 506,037 \rightarrow 4.873,866 \mu s$. Rasio setiap $10\times$ kenaikan N : $6,7\times, 12,7\times, 9,6\times$, rata-rata mendekati $10\times$, sesuai prediksi $O(n)$.

Konfirmasi $O(\log n)$ Binary Search: worst case comparisons untuk $N = 1.000, 10.000, 100.000, 1.000.000$ adalah 10, 14, 17, 20. Setiap $10\times$ kenaikan N hanya menambah $\sim 3-4$ comparisons, sesuai $\log_2(10) \approx 3,32$. Waktu eksekusi nyaris flat: $0,153 \rightarrow 0,159 \rightarrow 0,144 \rightarrow 0,216 \mu s$, variasi kecil ini lebih mencerminkan noise pengukuran dan cache behavior daripada pertumbuhan algoritmik.

Satu hal yang menarik: rasio speedup Binary Search vs Linear Search tidak konstan, melainkan terus membesar seiring N . Ini adalah manifestasi konkret dari fakta bahwa $O(n)/O(\log n) = O(n/\log n)$, yang merupakan fungsi yang bertumbuh, semakin besar N , semakin besar keuntungan Binary Search. Pada $N=1.000$ selisih $\sim 39\times$, pada $N=1.000.000$ selisih $\sim 22.564\times$.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-5-searching-wyattPNG>